AI6103 Homework: Effects of Learning Rate, Schedule, Weight Decay and Mixup on EfficientNetV2-S for Dog Breed Identification

Ayush Bhatt
Matric. No. G2610412E
Nanyang Technological University

1 Introduction

We train EfficientNetV2-S (Tan and Le 2021) from scratch on the Dog Breed Identification dataset (120 breeds; 7,155/1,533/1,534 train/val/test images) using SGD with momentum 0.9, implemented in PyTorch (Paszke et al. 2019). All hyperparameters are tuned on the validation set; the test set is used once at the end. All runs use batch size 128, seed 0, and the cross-entropy loss with label smoothing 0.1. Metrics are logged per epoch (training metrics are running averages over the augmented mini-batches of that epoch). Generative AI (Claude Sonnet 5.5 (Anthropic 2026)) was used to help prepare the LaTeX report, format tables and figures, and give feedback on draft text; all experiments, analysis and final wording are my own.

2 Data Preprocessing

Images are resized to 100×100. Channel statistics are computed on the training set (pixel values in $[0, 1]$) and used to normalise all splits (Table 1). Training images are augmented with a random horizontal flip ($p = 0.5$) and a random 100×100 crop of the image zero-padded by 12 pixels on all sides. The code is shown in Listing 1.

	R	G	B
Mean	0.4772	0.4528	0.3928
Std	0.2534	0.2480	0.2523

Table 1: Per-channel mean and standard deviation of the training set.

```
resize = v2.Resize((100, 100), antialias=True)
imgs = torch.stack([resize(decode_image(
    f'./data/train/images/{p}.jpg',
    mode=ImageReadMode.RGB))
    for p in trainDf.id]).to(device)  # uint8
x = imgs.float() / 255
mean = x.mean(dim=(0, 2, 3))
std = x.std(dim=(0, 2, 3))
norm = v2.Normalize(mean.tolist(), std.tolist())
PAD = 12

def augment(xb):  # xb: uint8 (B,C,H,W) on GPU
  B, _, H, W = xb.shape
  dev = xb.device
  flip = torch.rand(B, device=dev) < 0.5
  xb = torch.where(flip[:, None, None, None],
                   xb.flip(3), xb)  # h-flip, p=0.5
  xp = F.pad(xb, (PAD, PAD, PAD, PAD))  # zero pad
  dy = torch.randint(0, 2*PAD+1, (B,), device=dev)
  dx = torch.randint(0, 2*PAD+1, (B,), device=dev)
  rows = dy[:, None] + torch.arange(H, device=dev)
  cols = dx[:, None] + torch.arange(W, device=dev)
  b = torch.arange(B, device=dev)[:, None, None]
  out = xp[b, :, rows[:, :, None], cols[:, None, :]]
  return out.permute(0, 3, 1, 2)  # (B,C,H,W)

def prep(xb, train=False):
  if train: xb = augment(xb)
  return norm(xb.float() / 255)
```

Listing 1: Data preprocessing and augmentation.

3 Learning Rate

Three runs with initial learning rate 0.5, 0.05 and 0.001; no weight decay, no schedule, 15 epochs. Final-epoch results are in Table 2 and the curves in Figure 1.

LR	Train loss	Train acc	Val loss	Val acc
0.5	4.5881	7.98%	4.8432	5.41%
0.05	4.9521	3.42%	4.9880	3.52%
0.001	6.8610	0.73%	6.8901	0.52%

Table 2: Final (epoch 15) losses and accuracies for different initial learning rates.

The best learning rate for both validation and training, loss and accuracy, is lr=0.5 and they are Training-(7.98%, loss=4.5881) and validation-(5.41%, loss=4.8432). For training the main point which leads to lower loss and higher accuracy is being able to decrease the loss quickly, and at low number of epochs (15) this happens best at lr=0.5 as it allows us to take bigger steps towards lesser loss. For validation at this early training, low number of epochs, overfitting doesn't come into picture as all runs will be underfitted, from data we can see Δ(train$-$val) in accuracy is just 2.57 points for lr=0.5, so validation

Copyright © 2024, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

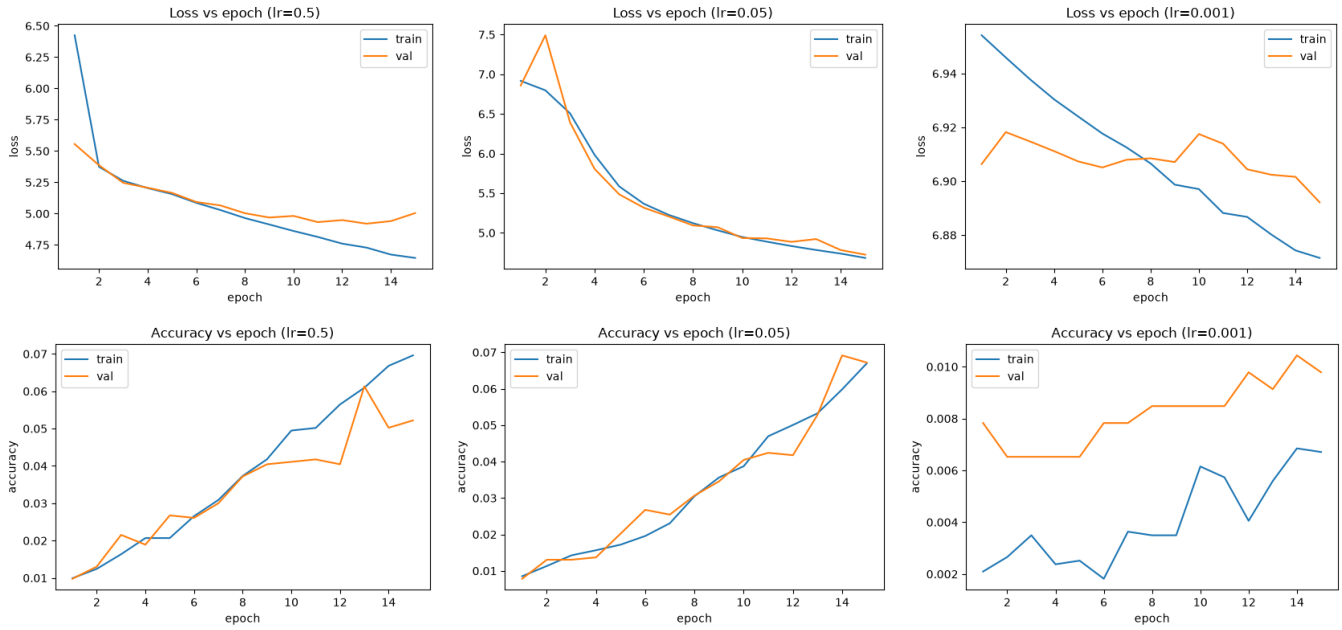


Figure 1: Training/validation loss (top) and accuracy (bottom) vs. epoch for initial learning rate 0.5, 0.05 and 0.001 (left to right), 15 epochs.

mainly depends on training loss and accuracy i.e. less training loss less validation loss. This explanation also holds for our other learning rates i.e. 0.05 which has lower accuracy and higher loss in both validation and training at train_acc=3.42%, train_loss=4.9521 and val_acc=3.52%, val_loss=4.9880. Same story goes with lr=0.001 as the step size is even smaller the results worsen, train loss 6.8610 and accuracy 0.73% and val loss 6.8901 and accuracy 0.52%.

4 Learning Rate Schedule

4.1 Cosine Annealing

Cosine annealing (Loshchilov and Hutter 2017) is a learning rate schedule. With initial learning rate η_0 and T total optimisation steps, the learning rate at step t is

$$\eta_t = \tfrac{1}{2}\eta_0 \left(1 + \cos\frac{\pi t}{T}\right), \qquad (1)$$

so η_0 decays smoothly to 0 at $t = T$ (Figure 2). Here $T = 300 \times \lceil 7155/128 \rceil = 16{,}800$ and the scheduler is stepped after every mini-batch. Cosine annealing is a learning rate scheduler which builds up on the step-down scheduler and removes the abruptness of step-down and makes it more smooth. As observed from Eq. 1. it decreases learning rate slowly at first, then fastest in the middle and lastly it slows down in lowering the learning rate. This helps because it makes the steps at start take bigger steps for faster drop and again slows the decreasing at end which helps in converging the weights near minimum.

4.2 Experiments

Using the best learning rate $\eta_0 = 0.5$ (best in Section 3), we train for 300 epochs with (1) a constant learning rate and (2) cosine annealing to zero. Results are in Table 3 and Figure 3.

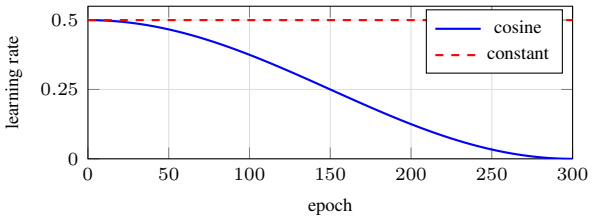


Figure 2: Learning rate schedules used in Section 4 ($\eta_0 = 0.5$).

Setting	Train loss	Train acc	Val loss	Val acc
Constant LR	1.0381	99.93%	5.5300	19.77%
Cosine	1.0469	99.93%	5.4822	20.74%

Table 3: Final (epoch 300) results with constant learning rate vs. cosine annealing ($\eta_0 = 0.5$).

According to Table 3, training loss (1.0381 vs 1.0469) and training accuracy (99.93% vs 99.93%) for constant learning rate vs cosine annealing are almost the same, and for validation cosine annealing is only slightly better, loss (5.5300 vs 5.4822) and validation accuracy (19.77% vs 20.74%). A constant learning rate of 0.5 stays large to the end, so the weights keep bouncing around from mini-batch noise and never settle. Cosine decays the rate to zero, so the last epochs take small steps and the weights settle into a minimum. That settled solution may generalise a bit better, giving slightly lower validation loss and higher accuracy, but the difference

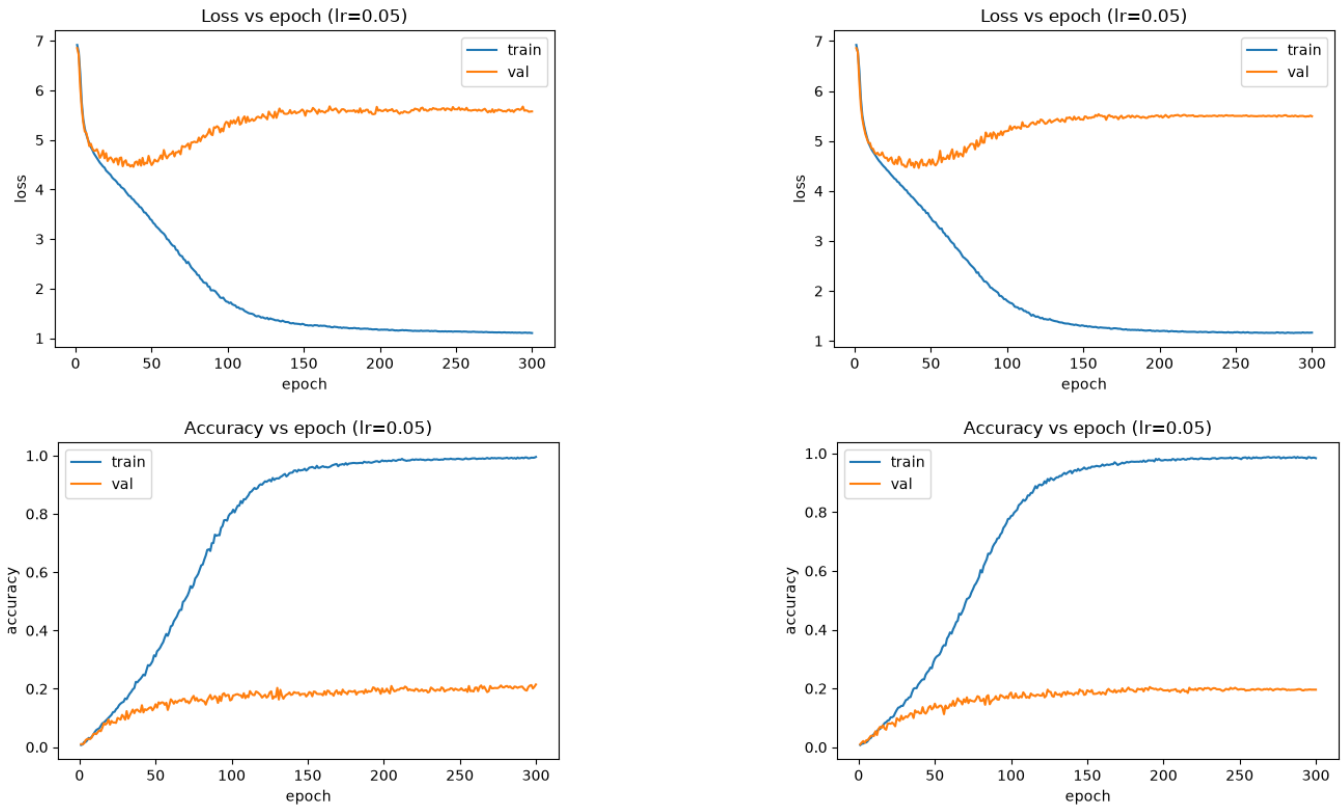


Figure 3: Training/validation loss (top) and accuracy (bottom) vs. epoch for a constant learning rate (left) and cosine annealing (right), $\eta_0 = 0.5$, 300 epochs.

is below one point of accuracy, which is small compared to run-to-run variation, so this is only weak evidence. It is a convergence effect, not a regularisation effect. Cosine does not stop overfitting, since both runs still overfit.

5 Weight Decay

Cosine annealing with $\eta_0 = 0.5$ and weight decay $\lambda \in \{5 \times 10^{-4}, 1 \times 10^{-4}\}$, 300 epochs. Results are in Table 4 and Figure 4.

λ	Train loss	Train acc	Val loss	Val acc
5×10^{-4}	1.0627	99.99%	3.9221	34.96%
1×10^{-4}	1.0416	99.94%	4.9891	24.98%

Table 4: Final (epoch 300) results for two weight decay coefficients.

From all our recent findings we can see that our model generally overfits our training data and thus the gap between training and validation accuracy is huge (Δ ($\sim$79 points), 99.93% vs 20.74% for cosine annealing). So we introduce a regularisation technique called weight decay. Weight decay is a technique which penalizes weights for being too large. This encourages them to be smaller and in turn it helps in generalizing since larger weights tend to help reach the values which might be exceptions, this does not solve overfitting completely just helps in reducing it (the gap is still 65 points with $\lambda = 5 \times 10^{-4}$). Thus after applying weight decay we can see a good amount of improvement relative(val_acc_cosine_decay_5 $-$ val_acc_cosine_decay_0) = $+68.6\%$ (relative diff here is between cosine annealing and $\lambda = 5 \times 10^{-4}$). This generalization just helps better validation without affecting our training accuracy. The larger weight decay here helps in better penalization of larger weights thus being better in validation accuracy which can be observed in Table 4: $\lambda = 5 \times 10^{-4}$ has val_accuracy 34.96% and $\lambda = 1 \times 10^{-4}$ has val_accuracy 24.98%, against 20.74% without weight decay. So even $\lambda = 1 \times 10^{-4}$ gives a clear gain (+20.4% relative), but the larger value is clearly better. Both have a similar training accuracy at about 99.9% (99.99% and 99.94%). These figures can also be seen in Figure 4. Similar trends as accuracies can be seen in loss in both training loss and validation loss for both $\lambda = 1 \times 10^{-4}$ and 5×10^{-4}. For $\lambda = 1 \times 10^{-4}$ the training loss barely changes from cosine annealing, 1.0469 without weight decay vs 1.0416 with it, but the validation loss improves from 5.4822 to 4.9891. With $\lambda = 5 \times 10^{-4}$ training loss slightly worsens and validation loss greatly improves at train_loss = 1.0627 and val_loss = 3.9221. We can see relative(train_loss_cosine_decay_5 $-$ train_loss_cosine_decay_0) = $+1.5\%$ (slightly

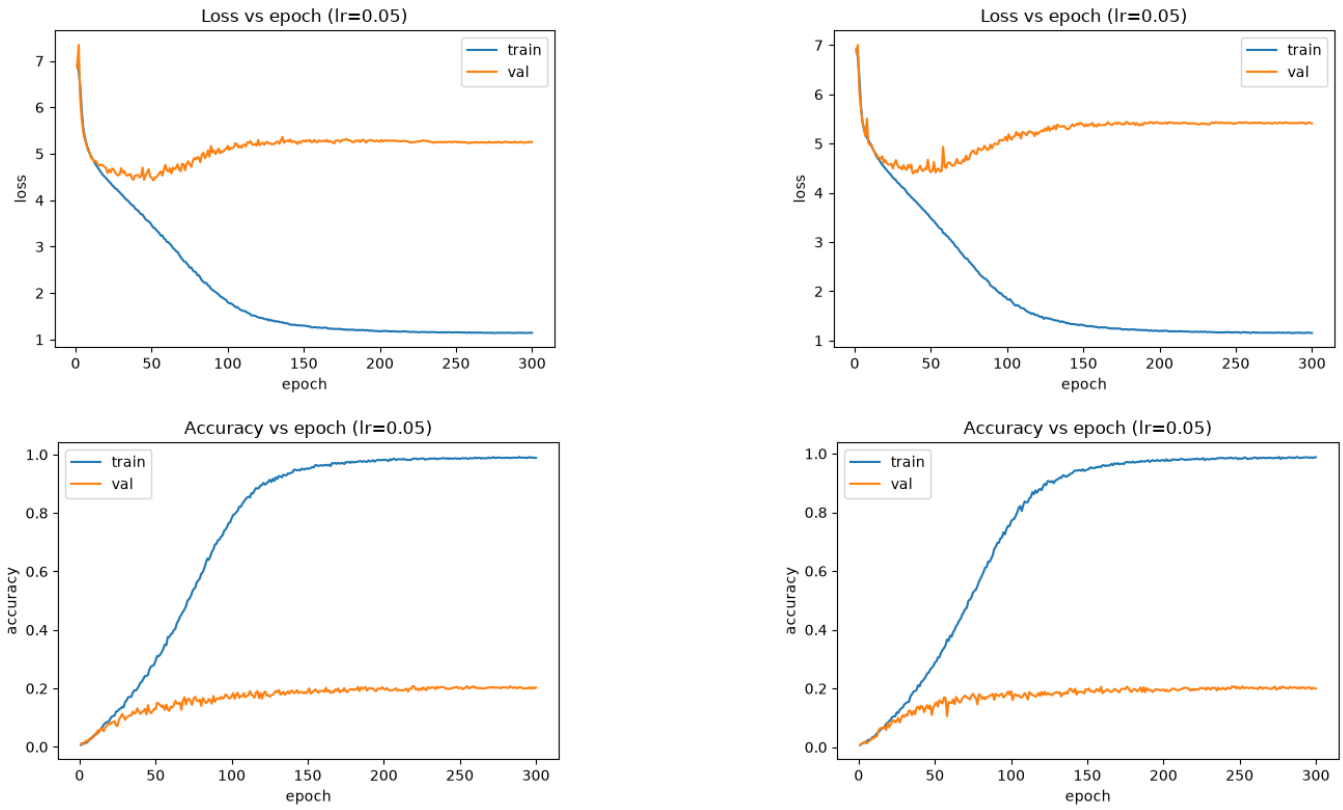


Figure 4: Training/validation loss (top) and accuracy (bottom) vs. epoch for weight decay $\lambda = 5 \times 10^{-4}$ (left) and 1×10^{-4} (right), with cosine annealing, 300 epochs.

worse) and relative(val_loss_cosine_decay_5 − val_loss_cosine_decay_0) = -28.5% (huge improvement).

6 Data Augmentation: Mixup

Mixup (Zhang et al. 2018) trains on convex combinations of random example pairs,

$$\begin{aligned} \tilde{x} &= \lambda x_i + (1-\lambda) x_j, \\ \tilde{y} &= \lambda y_i + (1-\lambda) y_j, \quad \lambda \sim \mathrm{Beta}(\alpha, \alpha), \end{aligned} \tag{2}$$

with $\alpha = 0.2$. The density of $\mathrm{Beta}(0.2, 0.2)$ is shown in Figure 5. We apply mixup on top of flip and crop augmentation, with the best setup so far (cosine annealing, $\eta_0 = 0.5$, $\lambda_{\text{wd}} = 5 \times 10^{-4}$, 300 epochs). Training accuracy is computed against the dominant class of the mixed label; validation uses the clean images. Results are in Table 5 and Figure 6.

Setting	Train loss	Train acc	Val loss	Val acc
Mixup ($\alpha = 0.2$)	1.9781	90.62%	3.6858	25.77%
Test accuracy		27.38%		

Table 5: Final (epoch 300) results with mixup and accuracy on the hold-out test set.

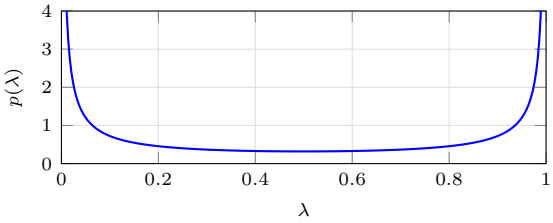


Figure 5: Probability density of $\mathrm{Beta}(\alpha{=}0.2, \beta{=}0.2)$, $p(\lambda) = \frac{\Gamma(0.4)}{\Gamma(0.2)^2} \lambda^{-0.8}(1-\lambda)^{-0.8}$.

From our previous findings with weight decay we know that regularisation is the right track to improve our overfitting problem. So we introduce another regularisation technique, mixup. Mixup is a technique that mixes raw input data with different labels to create a new input to be fed to our model during training. Mixup is expected to help by softening the decision boundary between labels, which can reduce overfitting (but does not completely eliminate it). This softening of decision boundary is meant to keep the model from tilting towards the noise and exceptions and keep it towards the generalized trend. With $\alpha = 0.2$ the mixing coefficient $\lambda \sim \mathrm{Beta}(0.2, 0.2)$ is usually close to 0 or 1 (Figure 5), so most mixed samples

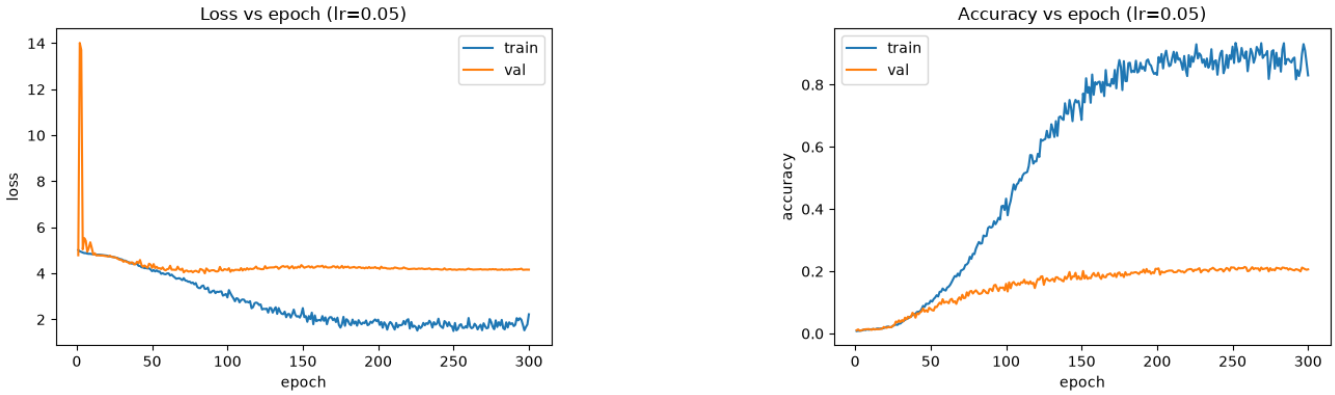


Figure 6: Training/validation loss (left) and accuracy (right) vs. epoch with mixup ($\alpha = 0.2$), 300 epochs.

stay close to one of the two original images, which keeps the regularisation mild. From Table 5, we can see that train_loss and train_acc between mixup and without mixup, i.e. using cosine with weight decay 5×10^{-4}, the difference is significant and we lose a lot of training accuracy (relative(train_acc_with_mixup $-$ train_acc_without_mixup) = -9.4%) and loss increases (relative(train_loss_with_mixup $-$ train_loss_without_mixup) = $+86.1\%$) as well. This worsened training accuracy occurs because we are mixing data and creating soft new labels which then gets compared against dominant class, this is why the training accuracy drops (the final-epoch training metrics with mixup also fluctuate noticeably between epochs, so these changes are approximate). For validation, on the other hand, we do not see an improvement in val_accuracy, which drops, and only a small reduction in val_loss, relative(val_acc_with_mixup $-$ val_acc_without_mixup) = -26.3% and relative(val_loss_with_mixup $-$ val_loss_without_mixup) = -6.0%. The gap between training and validation accuracy is essentially unchanged, 65.0 points without mixup (99.99 $-$ 34.96) and 64.9 points with it (90.62 $-$ 25.77), so mixup did not reduce our overfitting here. A possible reason is that with $\alpha = 0.2$ most mixes are mild, and on top of weight decay and cosine annealing it adds little. The final test accuracy is 27.38%, close to the validation accuracy of 25.77%.

References

Anthropic. 2026. Claude Sonnet 5.5 (model ID: claude-sonnet-5-5), accessed via Claude Code. Large language model, https://www.anthropic.com. Used for LaTeX report preparation, code and result formatting, and feedback on draft text. Accessed October 2026.

Loshchilov, I.; and Hutter, F. 2017. SGDR: Stochastic Gradient Descent with Warm Restarts. In *The 5th International Conference on Learning Representations*.

Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; Desmaison, A.; Köpf, A.; Yang, E.; DeVito, Z.; Raison, M.; Tejani, A.; Chilamkurthy, S.; Steiner, B.; Fang, L.; Bai, J.; and Chintala, S. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*.

Tan, M.; and Le, Q. V. 2021. EfficientNetV2: Smaller Models and Faster Training. In *Proceedings of the 38th International Conference on Machine Learning*.

Zhang, H.; Cisse, M.; Dauphin, Y. N.; and Lopez-Paz, D. 2018. mixup: Beyond Empirical Risk Minimization. In *The 6th International Conference on Learning Representations*.